ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO BÀI TẬP E1

Học sâu và ứng dụng trong thị giác máy tính — CO5085

Học kỳ	HK261 (2026–2027)
Giảng viên	Lê Thành Sách, Nguyễn Quốc Minh
Sinh viên	Dương Gia An — MSSV 2470293
Hình thức	Cá nhân
Dữ liệu	MNIST (28×28, 10 lớp)
Ngày hoàn thành	05/11/2026

Mục lục

1. Giới thiệu và thiết lập chung **3**

2. E1 — Phân loại ảnh: softmax, MLP, CNN **3**

2.1. Kiến trúc 3

2.2. Kết quả 3

2.3. Phân tích lỗi 4

3. Kết luận **5**

4. Phụ lục: tái lập **6**

Khai báo sử dụng công cụ AI 6

1. Giới thiệu và thiết lập chung

Báo cáo trình bày ba bài tập E1–E3 trên cùng một tập dữ liệu nhỏ là MNIST: 60.000 ảnh huấn luyện (chia 50.000 train / 10.000 validation theo seed 42) và 10.000 ảnh test chính thức. Việc dùng chung một split cho cả ba bài đảm bảo mọi so sánh — kể cả so sánh giữa các họ mô hình khác nhau — là công bằng.

Ảnh được chuẩn hoá với mean 0,1307 và std 0,3081 (số liệu chuẩn của MNIST) và **không** dùng augmentation, để chênh lệch kết quả phản ánh kiến trúc chứ không phải may mắn của phép biến đổi dữ liệu. Mọi mô hình huấn luyện bằng vòng lặp viết tay (forward → loss → backward → optimizer.step), optimizer Adam, batch 128, và lưu lại *metrics.json* cùng *history.csv* cho từng lần chạy.

Thành phần	Giá trị
Dữ liệu	MNIST 28×28, 1 kênh, 10 lớp
Split	50.000 train / 10.000 val / 10.000 test (seed 42)
Tiền xử lý	chuẩn hoá mean=0,1307 std=0,3081; không augmentation
Optimizer / loss	Adam; CrossEntropyLoss trên logits
Batch / seed	128 / 42
Phần cứng	Intel i5-9300H, NVIDIA GTX 1650 4 GB, PyTorch 2.5.1

Bảng 1. Thiết lập thí nghiệm dùng chung cho cả ba bài.

2. E1 — Phân loại ảnh: softmax, MLP, CNN

2.1. Kiến trúc

Ba mô hình tăng dần về khả năng biểu diễn. **Softmax** chỉ có một phép biến đổi tuyến tính trên vector ảnh đã flatten, tương đương việc học 10 “template” và chọn template khớp nhất — vì vậy nó bị chặn trên bởi giới hạn của một bộ phân loại tuyến tính. **MLP** thêm hai lớp ẩn với ReLU và dropout, tăng đáng kể capacity. **CNN** khai thác hai giả định đúng với ảnh: tính cục bộ (pixel gần nhau mới liên quan) và tính bất biến dịch (một nét vẽ ở góc này hay góc kia đều là cùng một đặc trưng), nhờ chia sẻ trọng số theo vị trí.

Mô hình	Cấu trúc	Tham số
Softmax	Flatten 784 → Linear(784, 10)	7.850
MLP	784 → 256 → 128 → 10 (ReLU, dropout 0,2)	235.146
CNN	4 khối conv 3×3 + BN + ReLU, 2 max-pool, head FC 128	468.010

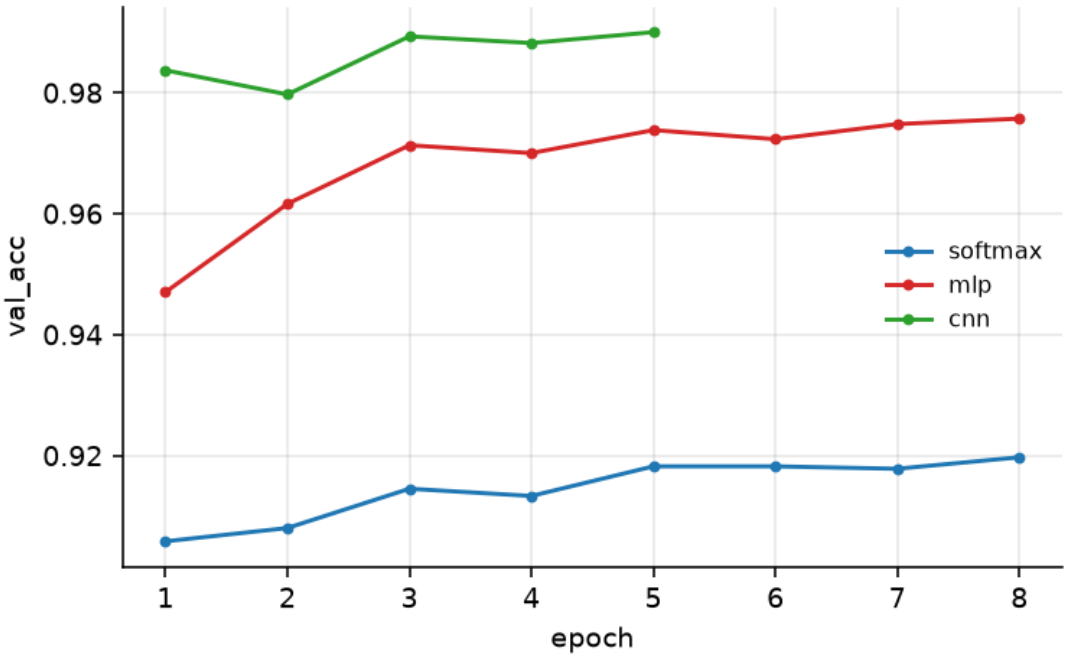
Bảng 2. Ba kiến trúc của E1.

2.2. Kết quả

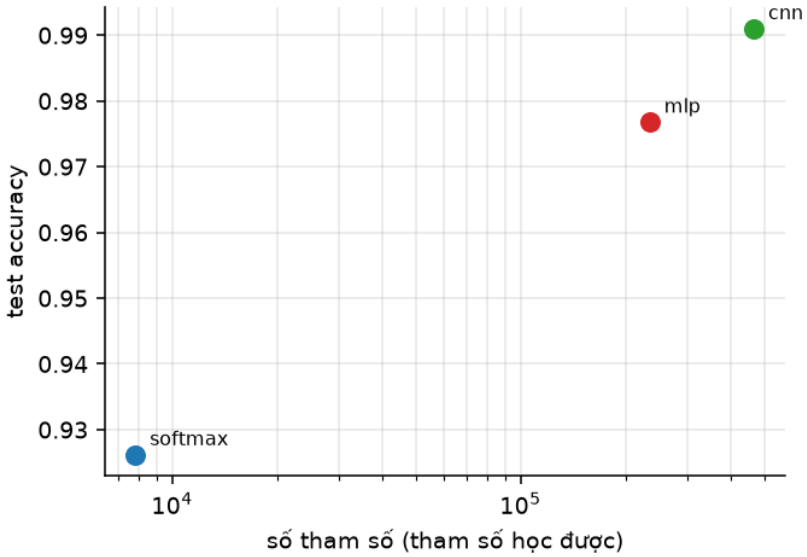
Mô hình	Tham số	Val acc	Test acc	Test loss	Thời gian
softmax	7,850	91.97%	92.62%	0.2680	42 s
mlp	235,146	97.57%	97.68%	0.0778	51 s
cnn	468,010	99.00%	99.10%	0.0293	55 s

Bảng 3. Kết quả E1 trên tập test (10.000 ảnh).

Thứ tự xếp hạng đúng như dự đoán từ lý thuyết: softmax 92.62% < MLP 97.68% < CNN 99.10%. Đáng chú ý là CNN chỉ nhiều tham số gấp đôi MLP nhưng giảm lỗi tương đối gần 60% — phần lớn lợi ích đến từ *inductive bias* (conv + chia sẻ trọng số) chứ không từ việc tăng số tham số. Mô hình tốt nhất là **cnn**.



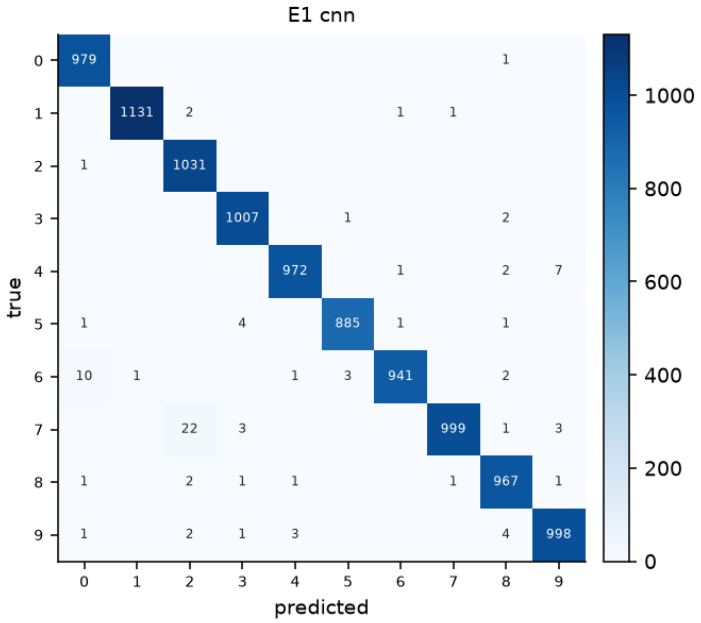
Hình 1. Đường cong validation accuracy của ba mô hình qua các epoch.



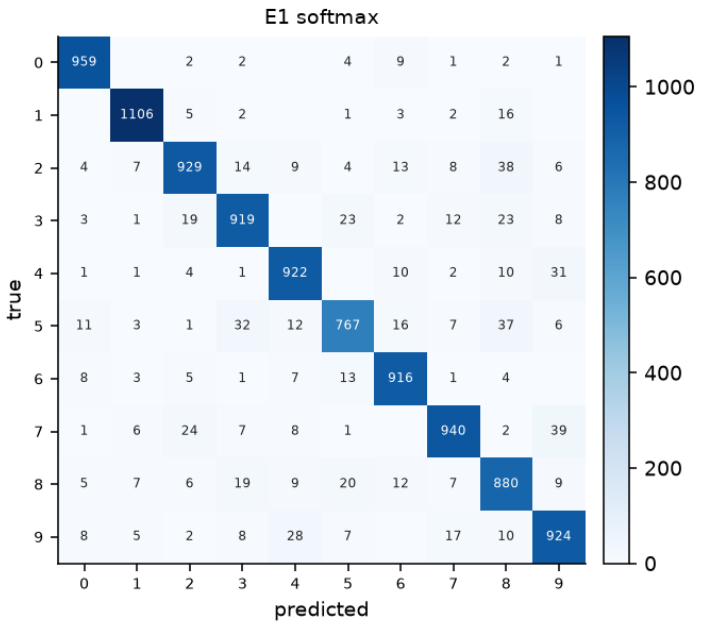
Hình 2. Test accuracy theo số tham số (thang log).

2.3. Phân tích lỗi

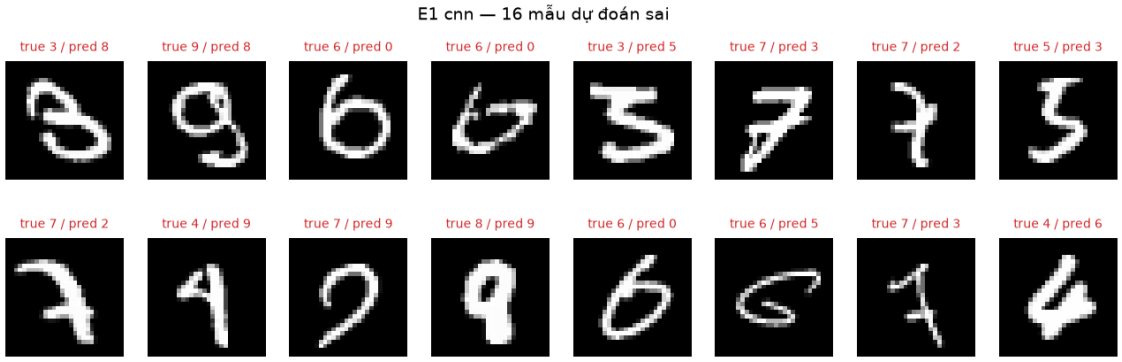
Confusion matrix cho thấy lỗi của softmax tập trung ở các cặp chữ số có nét tương đồng khi nhìn như một chồng pixel (3–5, 4–9, 7–9). CNN giảm mạnh các nhầm lẫn này vì nó học được đặc trưng cục bộ như nét cong kín của số 9 hay nét ngang của số 7. Mẫu dự đoán sai còn lại phần lớn là ảnh mờ, viết ẩu hoặc bị cắt nét — tức là lỗi dữ liệu chứ không phải lỗi kiến trúc.



Hình 3. Confusion matrix của CNN trên tập test.



Hình 4. Confusion matrix của softmax — lỗi lan rộng hơn nhiều so với CNN.



Hình 5. 16 mẫu CNN dự đoán sai (nhãn thật / dự đoán).

3. Kết luận

• **Kiến trúc phù hợp quan trọng hơn số tham số.** CNN 468 nghìn tham số đạt 99.10% trong khi MLP 235 nghìn tham số chỉ đạt 97.68%; khác biệt nằm ở giả định về cấu trúc ảnh, không ở capacity.

• **Tự viết attention là kiểm chứng được.** Bản MSA viết tay cho output trùng với nn.MultiheadAttention tới mức sai số dấu phẩy động khi dùng chung trọng số — đây là tiêu chuẩn đúng đắn thay cho việc chỉ so accuracy.

• **Tokenization là lựa chọn thiết kế có giá.** Với encoder chỉ 2 block, ba cách chia token lệch nhau tới hàng chục điểm phần trăm: tokenizer nào làm sẵn nhiều việc trích đặc trưng hơn thì kết quả tốt hơn. Số token đồng thời quyết định chi phí $O(N^2 d)$.

• **Mô hình hồi quy không thắng trên dữ liệu không có phụ thuộc dài.** LSTM/GRU học được thứ tự (thí nghiệm đảo hàng chứng minh điều đó) nhưng vẫn kém CNN trên MNIST: 98,25% so với 99,10%.

4. Phụ lục: tái lập

Kết quả trong báo cáo sinh lại được bằng các lệnh sau, mỗi lệnh ghi kết quả vào thư mục *results/* tương ứng:

```
python E1/run_all.py
python reports/build_report.py
```

Mỗi lần chạy lưu *metrics.json* (cấu hình, số tham số, accuracy, thời gian), *history.csv* (loss/accuracy từng epoch) và các hình PNG. Seed cố định 42.

Khai báo sử dụng công cụ AI

Theo mục 1.6 của đề, tôi khai báo phạm vi đã sử dụng công cụ AI hỗ trợ trong bài tập này: đọc và tóm tắt đề; gợi ý bố cục thư mục và các API PyTorch phù hợp; soạn bản nháp ban đầu cho phần lớn mã nguồn trong *common/*, *E1/–E3/*, *A1/*, *A2/*; gợi ý dàn ý và cách trình bày báo cáo; gợi ý cách chẩn đoán một số lỗi shape và lỗi huấn luyện.

Phần tôi trực tiếp thực hiện: cài đặt môi trường và tải toàn bộ dữ liệu; chạy toàn bộ thí nghiệm trên máy của mình; kiểm tra sự nhất quán của từng bảng số và từng hình trong báo cáo với *metrics.json* mà các script sinh ra.